第 2 周分工（11/9 – 11/15 23:59）

本周唯一目标：四块地基各长出"真东西"——数据表逐格核对完、环境镜像真构建过、第一批复现器跑起来、报告骨架能编译。

本周的硬事实（截至 11/14 19:20）：仓库里目前只有 B 的 7 条提交，A/C/D 本周 **0 提交**。距离检查还有约 **28 小时**，每人至少要留下 1 条实质提交，否则这一周记空白（漏了不可补）。

角色	成员	本周交付
A 数据与统计	xxt（学号 `<学号>`）	5 张表逐格核对 + deviations 表 + 追查缺失的 2 个 BoltDB bug
B 环境与可复现	张歆海 1240012291	镜像在 CI 与**本机**各构建一次 + 环境表填满 + digest 钉死（已完成，只剩本机复测）
C 复现实验	待定	10 个候选 bug + **3 个能稳定触发的复现器**
D 报告与统筹	待定	加 collaborator + 角色表填名 + weekly-log + Introduction 英文初稿

时间线

时间	动作
今天（周六）晚	各自开始动手（collaborator 上周已加好，无需再做）
周六 23:00	每人本地 `git commit` 完成，群里报一句
周日 12:00	全部 push（先 `git pull --rebase`）
周日 21:30	开 PR / 合并；确认 `main` 能跑
周日 23:59	助教检查

分支：`a-dataset`、`b-env`、`c-repro`、`d-report`。提交信息：`<scope>: <改了什么> (学号 姓名)`。

A — xxt（数据与统计）

目标：把"论文的每个数字"变成"仓库里能指到的一行"，并且把对不上的地方解释清楚。

要交的文件

文件	说明
`docs/tables_check.md`	**新建**：Paper vs Ours 逐格对照（Table 5/6/7/9/10），每格标 ✅ / ⚠️
`docs/deviations.md`	**新建**：deviations 表（条目 / 论文设置 / 我们的设置 / 原因）
`results/tables/*.csv`	由 `scripts/parse_dataset.py` 生成，**首次入库**（目前只在 B 的机器上未跟踪）
`docs/table_mapping.md`	复核列号（重点：Table 6 的 `Wait` = `wait related` + `WaitGroup` 两列相加）

步骤

1. clone 仓库，跑 `python scripts/fetch_dataset.py --check` → `python scripts/parse_dataset.py`，确认输出 `blocking=83 non-blocking=86 total=169`。
2. 打开论文 PDF 的 Table 5/6/7/9/10，逐格填进 `docs/tables_check.md`：论文值、我们的值、差值、判定。
3. 对 4 处 ⚠️ 写原因（现有线索见 `docs/claims.md` 的 C1/C2）：

- blocking 83 vs 85（BoltDB 少 2 条）
- Mutex 27 vs 28、Chan-with 15 vs 16（同一条原因）
- message passing 65 vs 66
- `kubernetes#11298` 同时标了 shared 和 message（+1 抵消了 −1）

1. **追查缺失的 2 条**：在 BoltDB 仓库的 commit 历史里搜 `392`、`240`、`deadlock`、`lock`，看能不能找到对应的修复补丁并补进分析；找不到就写明搜索范围和结论。
2. `docs/deviations.md` 写 4 条：Go 版本、复现规模（15–20 个 vs 41 个）、实验主机（GitHub runner）、数据集缺 2 条。

验收标准

- 干净机器 `python scripts/parse_dataset.py` 五秒内出结果，与入库 CSV 一致；
- `docs/tables_check.md` 每格都有论文值、我们的值、判定；
- 每个 ⚠️ 都有可验证的原因，不是"可能存在差异"这种空话。

提交

```
dataset: verify all recomputed tables cell by cell (<学号> xxt)
dataset: add deviations table and BoltDB gap analysis (<学号> xxt)
```

B — 张歆海 1240012291（环境与可复现）

目标：把实验环境钉成一个可重建的镜像，并且**至少有两次独立构建的实测数据**。

本周已完成（不用重做，只需知道现状）

项	状态
`Dockerfile`（base 镜像按 **digest** 钉死，go1.22.4 / go1.21.13 两版）	✅ 已入库
`.github/workflows/env.yml`：干净 Ubuntu runner 上构建 + 冒烟测试 + 把测量结果提交回仓库	✅ 已跑通两次，产物在 `results/ci/`
`docker/verify.sh`：一条命令测出内核/Go/gcc/digest/体积/构建耗时/`-race` 是否生效	✅ 已入库
`docs/environment.md`：L2 表已按实测填满	✅
`README`：预期输出与运行时间、可复现性对照表	✅

本周剩余（今天做完）

1. Docker Desktop 装完并重启后，本机跑一次 `bash docker/verify.sh`，把输出存成 `results/local/env-L2-<主机名>.md`；
2. `docs/environment.md` 增加一行"本机（Windows 11 + Docker Desktop）"的实测结果，作为第二个独立环境；
3. 对比两次结果：Go/gcc/镜像大小应完全一致，内核与构建耗时不同——把差异写进 deviations（这是**证据**，不是问题）。

验收标准

- `bash docker/verify.sh` 在本机通过，且 `results/local/` 里有实测文件；
- `docs/environment.md` 里两个宿主机的数字都对得上 `results/` 里的原始产物；
- 助教只看 README + environment.md 就能重建镜像，不需要问人。

提交

```
env: record a second independent build on the local Docker host (1240012291 张歆海)
```

C — 待定（复现实验）

目标：从 169 个 bug 里挑出 10 个可做的，并把其中 3 个真正跑起来。

要交的文件

文件	说明
`repro/candidates.md`	**新建**：10 个候选，每个写 6 项（仓库+commit、BugID、论文症状、触发方式、工作量、风险）
`repro/<app>-<id>/`	**新建至少 3 个目录**：`README.md` + `reproduce.sh` + `notes.md`
`docs/claims.md`	复核 C1–C4 措辞；C4 的"数据集标 19/23，论文写 21/20"必须保留

步骤

1. 读论文 §4（方法）、§5.3、§6.3，记住两个协议：blocking bug 要 **3/3 次触发**；race 检测器跑 **100 次**算命中。
2. 打开 `results/tables/bug_inventory.csv`（A 会入库；没有就先跑 `python scripts/parse_dataset.py` 生成），按三条规则筛：`reproduced_flag = 1`、`patch_size_lines ≤ 10`、症状是"挂住不退出"（可用 timeout 判定）。挑 10 个，至少覆盖 3 个仓库。
3. 挑最简单的 3 个写复现器：

- `reproduce.sh` 契约：退出码 0 = 复现到 bug，1 = 没复现，2 = 构建失败；
- 跑法：`docker run --rm -v "$PWD/repro/<bug>:/work" -w /work cs340-repro:go1.22 ./reproduce.sh`；
- 每次运行往 `runs.jsonl` 追加一行（`{"bug":..., "run":n, "verdict":..., "wall_s":...}`）。

1. 如果本机没有 Docker：用 Codespaces（仓库里已有 `.devcontainer/`，开箱带 Docker），或在 CI 里加一个 `repro` 工作流——两条路都可以，但要在 PR 里说明用的是哪条。

验收标准

- 3 个复现器能稳定给出判定（blocking 类 3/3 触发）；
- 每个复现器有 `notes.md` 记录"相对论文改了什么"（Go 版本不兼容时的改法必须写清）；
- `repro/candidates.md` 里 10 个候选都能在 `bug_inventory.csv` 里查到对应 commit。

提交

```
repro: shortlist 10 candidates from the dataset inventory (<学号> <姓名>)
repro: add 3 working reproducers for blocking bugs (<学号> <姓名>)
```

D — 待定（报告与统筹）

目标：把仓库变成"四个人的仓库"，并把报告骨架填到能编译、能读。

要交的文件

文件	说明
GitHub 仓库设置	✅ 去年已加好（上周完成），本周不用管
`docs/team_division.md`	角色表填真实姓名 + 学号
`README.md`	Team 表补齐四人
`docs/weekly-log.md`	**新建**：日期 / 成员 / 提交摘要 / 对应报告章节
`report/main.tex`	Introduction 英文初稿 + Setup 骨架（引用 `docs/environment.md` 与 deviations 表）

步骤

1. 建四条分支（`a-dataset` / `b-env` / `c-repro` / `d-report`）；collaborator 上周已加好，不用管。
2. 填角色表与 README Team 表（A 是 xxt，B 是张歆海 1240012291，C/D 填认领的人）。
3. `report/main.tex`：Introduction 三段——problem（Go 的并发模型与"消息传递更安全"的信念）、claims（C1–C4 各一句）、scope（L1 全表 + L2 15–20 个 bug + Extension）。能编译成 PDF 即可。
4. `docs/weekly-log.md` 补第 1 周与第 2 周的记录（第 1 周：B 的 5 条提交；第 2 周：按实际填）。

验收标准

- 周日 12:00 前，四个人的提交都能在 `main` 上看到；
- `report/main.tex` 编译通过（Overleaf 或本地）；
- weekly-log 与 `git log` 对得上（这是 contribution statement 的证据）。

提交

```
meta: invite collaborators and fill the role table (<学号> <姓名>)
report: add introduction draft and weekly log (<学号> <姓名>)
```

文件归属表（`git add` 前对一下）

文件 / 目录	归属
`scripts/parse_dataset.py`、`scripts/fetch_dataset.py`、`results/tables/`、`docs/table_mapping.md`、`docs/tables_check.md`、`docs/deviations.md`	A
`Dockerfile`、`docker/`、`.dockerignore`、`.gitattributes`、`.devcontainer/`、`.github/workflows/`、`.gitignore`、`docs/environment.md`、`docs/env-setup.md`、`scripts/check_env.py`、`Makefile`、`results/ci/`、`results/local/`	B
`docs/claims.md`、`repro/`	C
`docs/plan.md`、`docs/team_division.md`、`docs/week1_tasks.md`、`docs/week2_tasks.md`、`CONTRIBUTING.md`、`report/`、`docs/weekly-log.md`、README 的 Team 表	D

不要 `git add .`：B 之前一次 `git add -A results` 差点把 A 的 CSV 带进自己的提交里，已经回滚了。

风险与兜底

风险	处理
权限：collaborator 已加（上周）	若某人仍推不上来，让对方确认接受了邀请、且用的是**自己的** GitHub 账号
C 没有 Docker 环境	Codespaces（`.devcontainer/` 已就绪）或让 B 的机器代跑
A 追不到那 2 个 BoltDB bug	写明搜索范围与结论，作为 limitation，不硬凑
某人今天做不完	**提前说**，由搭档接手并在 PR 描述里写清"谁做了什么"；不要让本周空着
代理/网络导致 push 失败	B 已给 git 配了"仅 github.com 走 127.0.0.1:7890 代理"；直连不通时先确认代理开着